

Faculdade Cotemig

Fundamentos Teóricos da Computação

Relatório Técnico: Implementação de Máquinas Abstratas

Autores:

Paulo Vitor Martins de Castro

Gabriel Fonseca de Araujo

Belo Horizonte - MG

2026

1. Introdução

Este relatório técnico apresenta a modelagem teórica e a implementação prática de três máquinas abstratas fundamentais para a teoria da computação: o Autômato Finito Determinístico (AFD), o Autômato de Pilha (AP) e a Máquina de Turing (MT). O trabalho tem como objetivo traduzir os formalismos matemáticos das linguagens regulares, livres de contexto e sensíveis ao contexto para implementações funcionais na linguagem de programação C#. Através da construção destes simuladores, explora-se os limites de cada modelo computacional e seu respectivo poder de reconhecimento.

2. Fundamentação Teórica

As três máquinas implementadas operam com níveis crescentes de capacidade de memória e processamento:

Autômato Finito Determinístico (AFD)

Um AFD é o reconhecedor padrão para linguagens regulares. Ele é formalmente definido por uma 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$, onde Q é o conjunto de estados, Σ é o alfabeto, δ é a função de transição, q_0 o estado inicial e F o conjunto de estados de aceitação.

Exemplo de cálculo manual (Linguagem L_1 , entrada **ab):**

1. O autômato inicia em q_0 .
2. Lê o símbolo **a**, transição $\delta(q_0, a)$ leva para q_1 .
3. Lê o símbolo **b**, transição $\delta(q_1, b)$ leva para q_2 .
4. Como a cadeia terminou e $q_2 \in F$, a cadeia é ACEITA.

Autômato de Pilha (AP)

O AP estende o AFD ao incluir uma memória auxiliar do tipo LIFO (pilha), permitindo o reconhecimento de linguagens livres de contexto. É formalizado pela 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset)$, operando neste projeto com o critério de aceitação exclusivo por pilha vazia.

Exemplo de cálculo manual (Linguagem $L_2 = a^n b^n$, entrada **aabb**):

1. Início em q_0 , pilha contém o símbolo inicial Z_0 .
2. Lê **a**: transição empilha **X** (Pilha: **X**, Z_0).
3. Lê **a**: empilha **X** (Pilha: **X**, **X**, Z_0).
4. Lê **b**: transita para q_1 , desempilha **X** (Pilha: **X**, Z_0).
5. Lê **b**: mantém-se em q_1 , desempilha **X** (Pilha: Z_0).
6. Fim da cadeia em q_1 : realiza transição vazia (ϵ), desempilha Z_0 e ACEITA por esvaziar a pilha.

Máquina de Turing (MT)

A MT é um modelo universal de computação, equipada com uma fita infinita de acesso aleatório. É descrita pela 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$.

Exemplo de cálculo manual (Linguagem $L_4 = a^n b^n c^n$, entrada **abc**):

1. Em q_0 , lê **a**, escreve **X**, move à direita $\rightarrow q_1$.
2. Em q_1 , lê **b**, escreve **Y**, move à direita $\rightarrow q_2$.
3. Em q_2 , lê **c**, escreve **Z**, move à esquerda $\rightarrow q_3$.
4. Em q_3 , retrocede até achar o marcador **X**, avança uma célula à direita $\rightarrow q_0$.
5. Em q_0 , lê **Y** (pois os 'a's acabaram), passa para estado de verificação $\rightarrow q_4$.
6. Em q_4 , percorre a fita confirmando que restam apenas marcadores. Encontra branco **_** e atinge q_{accept} .

3. Descrição da Implementação

Parte 1: AFD

A modelagem do AFD em C# seguiu estritamente as definições da 5-tupla, explicitamente nomeadas no código. A função de transição δ foi construída utilizando uma estrutura de dados `Dictionary<(string, char), string>`, mapeando uma tupla contendo o estado atual e o símbolo lido diretamente para o estado de destino. Para lidar com transições onde a função δ não é total (entradas inválidas fora de Σ), a implementação valida a existência da regra pelo método

`ContainsKey`. Caso o caractere não conste no dicionário ou no alfabeto, o processamento da cadeia é imediatamente interrompido e a mesma é rejeitada.

Parte 2: Autômato de Pilha

Implementado fazendo uso nativo do objeto `Stack<char>` da linguagem C#. O algoritmo força que o estado de aceitação dependa puramente do esvaziamento estrutural da pilha, ignorando verificações de estado final. A validação do topo da pilha foi feita através do método `Peek()`, e a remoção com `Pop()`. Ao final da leitura da cadeia, se o autômato está no estado de verificação de término e apenas o símbolo Z_0 se encontra na pilha, uma transição em branco ocorre para limpá-la e validar o aceite.

Parte 3: Máquina de Turing

A fita infinita da Máquina de Turing foi estruturada através de um `Dictionary<int, char>`. Diferente de um Array clássico que engessaria o escopo de memória, as chaves numéricas do dicionário permitem posições negativas e positivas crescentes simulando fita ilimitada. A transição possui o formato complexo de mapear estado e símbolo lido em uma tupla indicando o `(novoEstado, novoSimbolo, direcao)`. Para garantir segurança contra malhas de execução contínua (loops infinitos), instaurou-se um mecanismo que aborta a verificação preventivamente ao atingir uma contagem limite de 1000 passos.

4. Respostas às Questões Reflexivas Exigidas

Reflexões - AFD (Parte 1)

- **Regularidade de L_1 :** A linguagem L_1 é classificada como regular pois sua verificação não requer contagem de instâncias complexas (ausência de memória profunda). Ela é satisfatoriamente descrita pela expressão regular associada $(a|b)^*ab$.
- **Justificativa dos Estados:** q_0 representa o estado passivo lendo prefixos ou reincidindo em quebras. q_1 guarda na memória temporária que a letra 'a' acaba de ser identificada. Finalmente, q_2 (estado de aceitação) consolida a leitura de 'b' logo após um 'a'.
- **Tratamento de entradas não totais:** Sendo a função de transição parcial por natureza no código, o recebimento de qualquer símbolo não previsto ou combinação sem regra no `Dictionary` engatilha um falso na lógica de reconhecimento, rejeitando a string graciosamente.

Reflexões - AP (Parte 2)

- **Limitações e Lema do Bombeamento:** Um AFD é incapaz de reconhecer $L_2 = a^n b^n$ porque exigiria infinita quantidade de estados discretos para contar todo n concebível. Pelo Lema do Bombeamento, se partirmos uma string $w = xyz$ puramente formada de caracteres 'a', seu bombeamento para xy^2z quebraria instantaneamente a igualdade exigida entre as frequências de 'a' e 'b'.
- **Equivalência da Aceitação:** Na computação teórica, o aceite por estado final e por pilha vazia detêm poderes matematicamente rigorosos e equivalentes. Todo autômato de pilha vazia pode sofrer a inserção sintética de um novo símbolo no fundo que, quando avaliado, engatilha um pulo para um estado declarativo final.
- **Papel de Z_0 :** Z_0 atua como sentinela primordial. O seu uso impede o AP de desencadear falhas de operação ao tentar realizar um **Pop** numa estrutura vacante sem controle lógico explícito.

Reflexões - MT (Parte 3)

- **Propriedades Exclusivas da MT:** A linguagem $L_4 = a^n b^n c^n$ não é passível de análise em Autômatos de Pilha devido à operação limitante LIFO (Last-In, First-Out). Ao desempilhar os valores temporários 'b' em combate ao histórico de 'a', o AP consome integralmente seu contexto inicial, chegando desprovido de referências para contabilizar e equiparar a parcela respectiva de 'c'. A MT suprime este obstáculo munida da habilidade inata de reverter seu cabeçote direcional e ler a fita infinitas vezes.
- **Estratégia de Marcação:** Foram necessários exatos 5 estados operacionais (q_0 a q_4) além de 2 estados terminadores explícitos (q_{accept} , q_{reject}) totalizando 7 estados. A heurística consistiu em ler as posições originais transmutando **a** para **X**, **b** para **Y** e **c** para **Z**, com regressões a cada finalização de ciclo (1 a, 1 b, 1 c), invalidando se a ordem ou proporção colidir.
- **Tese de Church-Turing:** Computadores modernos, englobando as CPUs de múltiplos núcleos, não sobrepujam em poder e alcance semântico as Máquinas de Turing hipotéticas no quesito estrito do que é "computável". Tais construtos contemporâneos ganham somente em agilidade e densidade. Tudo aquilo exequível nas máquinas atuais é reproduzível na rudimentar e elegante fita de Turing.

5. Resultados e Testes

Máquina / Linguagem	Cadeia Testada	Resultado Esperado	Resultado Obtido
AFD (L_1)	ab, bab, ababab	ACEITA	ACEITA
AFD (L_1)	aab, ba, b, vazio	REJEITA	REJEITA
AP (L_2)	ab, aabb, aaabbb	ACEITA	ACEITA
AP (L_2)	aab, abb, ba, abab, vazio	REJEITA	REJEITA
MT (L_4)	abc, aabbcc, aaabbbccc	ACEITA	ACEITA
MT (L_4)	aabbc, ab, abcabc, vazio	REJEITA	REJEITA
MT ($f(n)=n+1$)	1, 11, 111, 11111	11, 111, 1111, 111111	SUCESSO

6. Análise Comparativa

Ao colocar os modelos lado a lado, evidencia-se a estrita correspondência à Hierarquia de Chomsky. O **AFD** demonstrou alta fluidez e simplicidade, mas revelou incapacidade massiva perante qualquer imposição em registrar frequências ou profundidades de informações. O **Autômato de Pilha** cobriu tais lacunas agregando uma memória linear de histórico (sendo o repositório adequado de contagem $a^n b^n$), com sua eficiência contudo condicionada à irreversibilidade do acesso à memória de forma arbitrária. A **Máquina de Turing**, no ponto mais alto, exibiu resiliência plena para linguagens entrelaçadas como L_4 e computação de funções puras (como exemplificado em $f(n)=n+1$), a custo substancial de uma proliferação de estados e da prolixidade mecânica para administrar movimentações de ida e volta do cabeçote na fita.

7. Conclusão

A tradução dos rudimentos matemáticos que embasam o cálculo em matrizes programáticas na linguagem C# clarificou, com riqueza de materialidade, a abstração da teoria. Ao presenciar as instâncias falharem ou esbarrarem nos "limites dos passos", a vulnerabilidade algorítmica de loops infinitos (e consequentemente os pressupostos do Problema da Parada) tornou-se muito palpável. A execução demonstrou que a elegância do AP e o poder titânico da Máquina de Turing advêm não de elementos complicados, mas de comandos atômicos precisos orientados a blocos simples de informações.

8. Referências

1. SIPSER, Michael. *Introduction to the Theory of Computation*. 3. ed. Boston: Cengage, 2013.
2. HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. *Introdução à Teoria de Autômatos, Linguagens e Computação*. 2. ed. Rio de Janeiro: Campus/Elsevier, 2003.
3. MENEZES, Paulo Blauth. *Linguagens Formais e Autômatos*. 6. ed. Porto Alegre: Bookman, 2010.